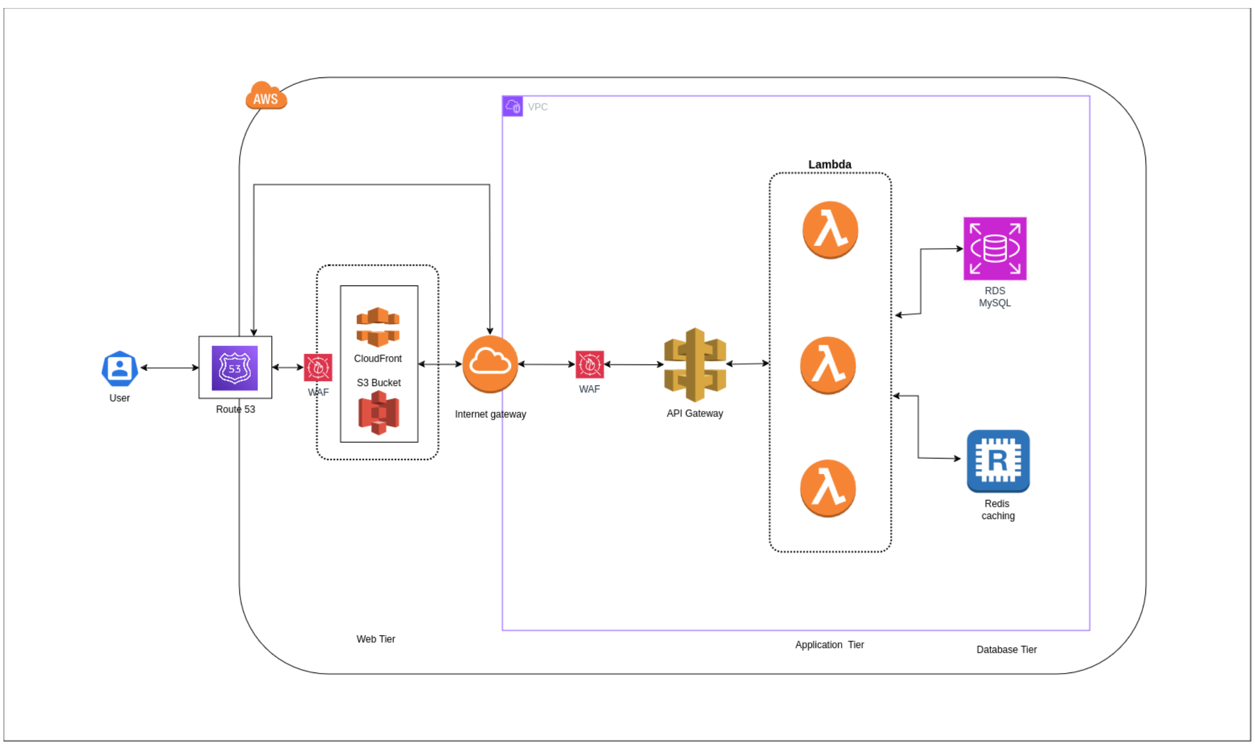
Architecture Design

Scenario: For event-driven, serverless applications with unpredictable traffic, **AWS Lambda** provides an auto-scaling, cost-efficient solution without managing infrastructure. Integrated with **API Gateway**, it handles incoming requests dynamically, scaling instantly based on demand. This setup is ideal for lightweight APIs, background tasks, and event-driven workloads where high availability and cost optimization are key.

- **Frontend**: Amazon S3 + Amazon CloudFront (CDN) + AWS WAF
- **Backend API**: Amazon API Gateway + AWS Lambda
- **Database**: Amazon RDS for MySQL
- **Caching**: Amazon ElastiCache for Redis
- **Security & Availability**: AWS WAF + Route 53
-

Infrastructure Architecture Diagram



Justification of Service Choices

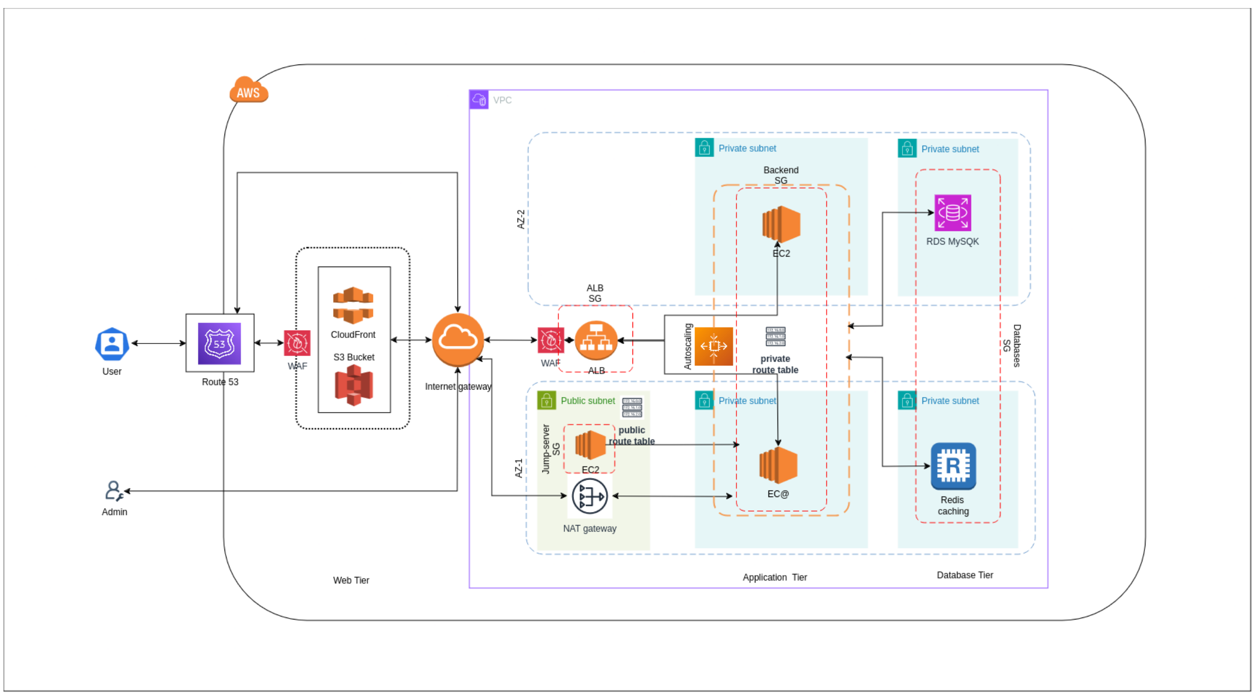
1. **Amazon Route 53**
    a. Provides highly available and scalable DNS routing for domain resolution.
    b. Supports health checks and traffic routing across AWS Regions.
2. **AWS WAF**
    a. Protects against SQL injection, XSS, and other web threats.
    b. Integrates with CloudFront and API Gateway for enhanced security.
3. **Amazon CloudFront (CDN)**
    a. Speeds up content delivery worldwide, reducing latency.
    b. Enhances security with AWS Shield integration.
4. **Amazon S3 (Static Website Hosting)**
    a. Stores and serves the frontend with high durability.
    b. Scales automatically to handle traffic spikes.
5. **Amazon API Gateway**
    a. Manages API requests efficiently with rate limiting and authentication.
    b. Directly integrates with AWS Lambda for a serverless backend.
6. **AWS Lambda**
    a. Provides a scalable, serverless backend without managing infrastructure.
    b. Automatically scales based on incoming API requests.
    c. Charges only for execution time and memory used.
7. **Amazon RDS (MySQL)**
    a. Managed relational database service with Multi-AZ support.
    b. Provides automated backups, patching, and monitoring.
8. **Amazon ElastiCache for Redis**
    a. Reduces database load by caching frequently accessed data.
    b. Improves application response time with in-memory data storage.

Architecture Design

Scenario: When building a high-traffic, stateful web application that requires persistent connections, low-latency performance, and custom configurations, AWS **ALB (Application Load Balancer), ASG (Auto Scaling Group), and EC2 instances** provide a scalable and resilient solution. ALB distributes traffic efficiently across EC2 instances, while ASG ensures automatic scaling based on demand, optimizing performance and cost. This setup is ideal for applications requiring continuous uptime, large compute resources, and advanced networking.

- **Frontend**: Amazon S3 + Amazon CloudFront (CDN) + AWS WAF
- **Backend API**: Application Load Balancer ,Auto-Scaling and ec2
- **Database**: Amazon RDS for MySQL
- **Caching**: Amazon ElastiCache for Redis
- **Security & Availability**: AWS WAF + Route 53

Infrastructure Architecture Diagram



Justification of Service Choices

1. **Amazon Route 53**
    a. Provides highly available and scalable DNS routing for domain resolution.
    b. Supports health checks and traffic routing across AWS Regions.
2. **AWS WAF**
    a. Protects against SQL injection, XSS, and other web threats.
    b. Integrates with CloudFront and API Gateway for enhanced security.
3. **Amazon CloudFront (CDN)**
    a. Speeds up content delivery worldwide, reducing latency.
    b. Enhances security with AWS Shield integration.
4. **Amazon S3 (Static Website Hosting)**
    a. Stores and serves the frontend with high durability.
    b. Scales automatically to handle traffic spikes.
5. **Application Load Balancer (ALB)**
    a. Distributes incoming traffic across EC2 instances.
    b. Ensures high availability by redirecting requests to healthy instances.
6. **Auto Scaling Group (ASG)**
    a. Dynamically scales EC2 instances **up or down** based on demand.
    b. Ensures cost efficiency by running only the necessary number of instances.
7. **Amazon EC2 (Backend API)**
    a. Provides a customizable, scalable environment for hosting the API.
    b. Suitable for applications requiring persistent connections and high compute resources.
8. **Amazon RDS (MySQL)**
    a. Managed relational database service with Multi-AZ support.
    b. Provides automated backups, patching, and monitoring.
9. **Amazon ElastiCache for Redis**
    a. Reduces database load by caching frequently accessed data.
    b. Improves application response time with in-memory data storage.